Frontend Specifications For AIPoPh

This document aims to be a concise representation of the specifications required to build a minimal working frontend for AIPoPh. The goal of this frontend product will be to demonstrate the user experience as they progress through the testing process.

Contract Setup

All contract-related requirements will be handled by the smart contract development team. In this demo, we will assume that the group running the testing framework controls everything with a single threshold NFT. This will model a centralized version of the hyperstructure without any loss of generality. This will simplify the backend and frontend development cycle tremendously.

Backend Setup

The backend team must develop the contract monitoring via the NEWM chain, test generation and verification, oracle updating, and transaction building. The contracts are still in the review phase thus the transaction building should be the last part implemented. The frontend team will need to mock these interactions until the contracts are finalized and the transactions required for the test to progress are finalized.

Frontend Setup

The frontend team will need to create multiple pages to handle various aspects of the testing process. The most complex part is the actual testing phase as this will require an interacting page where classical CAPTCHAs and AIPoPh meet. To simplify this interaction, the initial version should just represent the actual coloring of the graph as described below. Additional verification modules like reCAPTCHA, video calls, and mouse tracking may be added at a later time and should be considered out of scope for the demonstration.

Landing Page

The landing page will be a simple web3 login. We need a user to be able to connect their Cardano wallet, like Eternl, to the website. Once a user connects their wallet, we will have access to their base address and UTxO set. At this stage, we need to make sure that the user has enough Lovelace and NEWM to proceed. There will need to be a single button that begins the entire process. This will build an entry transaction into the proof of humanity test contract. They will need to place onto that UTxO the deposit, two units of incentive, and two units of fee for the automation. When the transactions land on the chain, the testing process begins. The user will need to be redirected to the testing process page.

Testing Process Page

At this point, the user has a UTxO inside the PoH contract waiting for a trusted actor from the testing facility to run the StartTest endpoint. This transaction will take a unit of incentive and a unit of fee and will add the `Begin The Proof Of Humanity Test` NFT to the UTxO. As well as the value change, the datum will be updated with the test data generated by the random number oracle via the backend. This transaction is a multi sig between the the user and the trusted actor holding the threshold NFT as defined in the Contract Setup section above. The trusted actor will build the transaction and will present it to the user for a signature. Once this transaction hits the chain the test is live so this transaction must have reasonable validity ranges to prevent troll attacks resulting in lost deposits.

When the test is live, the user will have a finite amount of time to enter the test page, complete the test, have the trusted actor sign off on the test, and submit the EndTest endpoint.

The Test Page

To ensure that a user takes the test, the Question generated will act as a hash for a sign-in. The user will have to sign the hash of the graph in the test to begin the test. Upon successful signature verification, the test will begin, allowing the user to color a graph presented to them. Once they have finished coloring the graph, a trusted actor will sign the hash of their coloring to ensure that the user ended the test properly. Both of these signatures are required inside the Answer type for the Question. The first signature proves that the user showed up to take the test, and the second signature proves that the test was taken in some valid form.

This is one of the most complex pages. It will require multiple cip8 signatures via a wallet and a valid ed25519 hotkey using an external non-cardano ed25519 signature library. It also requires an interacting graph that is game-like where a user can click on nodes to change the colors and a verification process that will check if their coloring is valid. As incorrect an Answer to the Question results in a lost deposit. This entire test is timed and if possible the remaining time is displayed in bold friendly letters.

Upon successful entry, coloring, and signature of the test, the user will create a transaction that calls the EndTest endpoint. This will place the UTxO of their Answer to the Question back into the PoH contract. At this point, the backend will take over to perform the test verification and certificate minting process. Failed tests have their deposits removed and remaining funds sent back to the user and successful tests have the `Congratulations! You're a human.` certificate of humanity minted into the CoH contract.

If the user fails the test then they should be redirected back to the landing page as they will have to retry the testing process, or else they proceed to the

Certificate of Humanity viewing page.

Certificate of Humanity Viewing Page

A user that passes the proof of humanity test will be issued a certificate of humanity that is held inside the CoH contract. The page needs to display the certificate in a nice way and a single button that will call the Verify endpoint. This page is open to everyone and all times. The point of the CoH contract is to hold valid and maybe invalid certificates so other users can verify them via on-chain and off-chain means.

There are multiple ways to prove that a certificate is valid. One is simply running the Verify endpoint. Anyone can run it. It is designed to be run from another contract as a verification method or it can be simply referenced from another contract such that the Wallet information can be used for signature verification.

Advance Tests For Frontend and Backend

The goal of the demonstration is to follow the happy path but there are many ways for this test to end in very unhappy ways. A user can fail to submit their test in time with the EndTest endpoint resulting in a lost deposit. In this state, the user will need to click the remove button themselves. A user may not leave during the test period or the test verification process. As stated above, invalid tests will lose their deposit. This may happen because the Answer doesn't answer the Question or it can happen because the trusted actor signing off on a completed test signs bad data. The trusted actor here can make a user forfeit the test. This may be due to many reasons like bot activity or troll attacks.

A user may lose their certificate of humanity. A certificate may have a validity range attached to it. If a certificate reaches its end of the validity range then it may be revoked by the trusted actor. A certificate can be revoked at any time for any reason by the trusted actor since they control the testing system.